

Travail Pratique (TP)

Conteneurisation, Orchestration et Pipeline CI/CD

d'une application Python (System Metrics Agent)

Application fournie	system_metrics_agent (Python / FastAPI / psutil)
Compétences visées	Docker, Docker Compose, GitHub Actions, Docker Hub, CI/CD
Modalité	Individuel
Livrables	Dépôt GitHub + README.md + images publiées sur Docker Hub
Durée indicative	1 semaine de travail personnel
Lien Github	https://github.com/Mficius/metrics_agent.git

1. Contexte et objectifs pédagogiques

Ce TP a pour but de mettre en pratique les fondamentaux du DevOps sur un cas concret : une application Python existante, fournie que vous allez conteneuriser, orchestrer et intégrer dans un pipeline d'intégration et de déploiement continu (CI/CD).

À l'issue de ce TP, vous serez capable de :

- Écrire des **Dockerfiles** distincts pour un environnement de développement et un environnement de production.
- Orchestrer plusieurs services applicatifs avec **Docker Compose**.
- Créer et publier des images sur un registre **Docker Hub**.
- Mettre en place un pipeline **GitHub Actions** réalisant automatiquement le build, les tests et la publication (push) d'images.
- Déployer une application conteneurisée à partir des images publiées.
- Documenter correctement un projet DevOps via un **README** professionnel.

2. Présentation de l'application fournie

L'application fournie, **system_metrics_agent**, est un agent Python modulaire qui collecte des métriques système (CPU, mémoire, charge système) et les transmet à une API de réception. Elle est composée de deux processus applicatifs distincts qui devront être conteneurisés :

- **Le service API** (`app.api`) : une API **FastAPI** qui expose les routes `/health`, `/metrics` (GET/POST) et `/metrics/latest`, et qui est lancée avec `uvicorn`.
- **Le service Agent** (`app.agent`) : un processus qui collecte les métriques toutes les `COLLECTION_INTERVAL` secondes et les envoie en HTTP vers `METRICS_ENDPOINT`.

Structure du projet fourni :

```
system_metrics_agent/
|-- app/
|   |-- __init__.py
|   |-- agent.py           # orchestration : collecte -> format -> envoi
|   |-- api.py             # API FastAPI de reception des metriques
|   |-- collector.py       # collecte CPU / RAM / charge (psutil, subprocess)
|   |-- config.py          # configuration via variables d'environnement (.env)
|   |-- formatter.py       # mise en forme du payload JSON
|   |-- sender.py          # envoi HTTP des metriques
|-- tests/                 # tests automatisés (pytest)
|-- .env.example           # modele de configuration
|-- requirements.txt
`-- README.md
```

La configuration se fait via un fichier `.env` (variables `METRICS_ENDPOINT`, `COLLECTION_INTERVAL`, `REQUEST_TIMEOUT`). Le projet contient déjà une suite de tests `pytest` que votre pipeline CI devra exécuter.

Vous ne devez pas modifier la logique métier de l'application (dossier `app/`) sauf si cela est strictement nécessaire à la conteneurisation. Le travail porte sur l'infrastructure DevOps autour de l'application, pas sur son code fonctionnel.

3. Prérequis

- Un compte **GitHub** actif, avec Git installé et configuré localement.
- Un compte **Docker Hub** (`hub.docker.com`) — à créer si vous n'en avez pas. C'est sur ce registre que vos images Docker seront publiées et téléchargées.
- **Docker Desktop** (ou Docker Engine + Docker Compose) installé et fonctionnel sur votre machine.
- Un éditeur de code (VS Code recommandé) et des notions de base de YAML.

- Avoir suivi les séances de cours sur Docker et l'intégration continue.

4. Travail demandé

Le travail est organisé en quatre parties progressives. Chaque partie s'appuie sur la précédente : il est recommandé de les traiter dans l'ordre.

Partie 1 — Conteneurisation de l'application (Dockerfiles)

1.1 Fichier `Dockerfile.dev` (environnement de développement)

Ce fichier doit permettre de développer confortablement à l'intérieur du conteneur. Il doit :

- Partir d'une image Python officielle adaptée (ex. `python:3.12-slim`).
- Installer toutes les dépendances, y compris celles de développement/test (`pytest` inclus).
- Permettre le **rechargement à chaud** du code (montage d'un volume + option `--reload` d'uvicorn pour le service API).
- Ne pas exécuter le code en tant qu'utilisateur `root` si cela reste simple à mettre en œuvre.

1.2 Fichier `Dockerfile` (environnement de production)

Ce fichier doit produire une image légère, sécurisée et optimisée pour la production. Il doit :

- Utiliser un **build multi-stage** (une étape de build/installation des dépendances, une étape finale minimale).
- N'installer **que** les dépendances nécessaires à l'exécution (pas `pytest` en production).
- Exécuter l'application avec un **utilisateur non-root** dédié.
- Définir un `HEALTHCHECK` pertinent (par exemple sur `/health` pour l'API).
- Exposer le port utilisé par l'API (`EXPOSE`) et définir une commande de démarrage claire (`ENTRYPOINT/CMD`).
- Minimiser la taille de l'image finale (choix de l'image de base, nettoyage du cache pip, `.dockerignore` soigné).

Astuce : comme le projet contient deux processus (API et agent), vous pouvez soit construire une seule image capable de lancer l'un ou l'autre selon une variable/commande, soit argumenter et documenter le choix d'une image unique partagée entre les deux services définis dans `docker-compose`. Les deux approches sont acceptées si elles sont justifiées dans le `README`.

Partie 2 — Orchestration avec Docker Compose

Créer un fichier `docker-compose.yaml` qui définit au minimum :

- Un service **api** (basé sur votre image de production) exposant le port de l'API sur l'hôte.
- Un service **agent** qui dépend du service **api** (`depends_on`) et qui lui envoie ses métriques via le réseau interne Docker (attention à l'URL utilisée dans `METRICS_ENDPOINT` : ce n'est plus `127.0.0.1` entre conteneurs).
- L'utilisation d'un fichier `.env` (`env_file`) pour la configuration, sans jamais committer de vraies valeurs sensibles.
- Un réseau Docker dédié reliant les deux services.
- Un `healthcheck` sur le service API, réutilisable par Compose.

Bonus (facultatif) : proposer un second fichier `docker-compose.override.yaml` permettant de basculer facilement en mode développement (montage de volumes, image `Dockerfile.dev`).

Partie 3 — Intégration continue avec GitHub Actions

Mettre en place un workflow GitHub Actions (fichier `.github/workflows/ci-cd.yml`) qui s'exécute automatiquement à chaque `push` (et idéalement à chaque `pull request`) sur la branche principale, et qui réalise les étapes suivantes, dans l'ordre :

- **Checkout** du code du dépôt.

- **Build** de l'image Docker de production à partir du `Dockerfile`.
- **Test** : exécution de la suite `pytest` du projet (dans le conteneur ou dans l'environnement du job) — le pipeline doit **échouer** si un test échoue.
- **Push** de l'image vers Docker Hub, **uniquement si les tests sont passés**, avec au minimum deux tags : le tag `latest` et un tag correspondant au SHA du commit (ou à un numéro de version).

Les identifiants Docker Hub ne doivent **jamais** apparaître en clair dans le code : ils doivent être stockés comme **secrets GitHub** du dépôt (Settings → Secrets and variables → Actions), typiquement `DOCKERHUB_USERNAME` et `DOCKERHUB_TOKEN` (un *access token* Docker Hub, pas votre mot de passe).

Vous êtes libres d'utiliser les actions officielles `docker/login-action`, `docker/setup-buildx-action` et `docker/build-push-action`, ou d'écrire les étapes en commandes `docker` classiques. Les deux approches sont acceptées.

Partie 4 — Publication et déploiement

- Créer un compte **Docker Hub** personnel (gratuit) si ce n'est pas déjà fait.
- Créer un (ou deux) **dépôt(s) d'images** sur Docker Hub pour ce projet (public ou privé selon vos consignes).
- Vérifier que votre pipeline CI publie effectivement une image consultable sur votre profil Docker Hub.
- Faire la démonstration d'un **déploiement** à partir des images publiées, sans utiliser le code source local : sur votre machine, exécuter `docker compose pull` puis `docker compose up` en pointant vers les images de Docker Hub (et non un build local).
- Fournir une preuve de fonctionnement (capture d'écran des conteneurs actifs, et d'un appel réussi à `/health` ou `/metrics/latest`).

Bonus (facultatif, points supplémentaires) : déployer réellement l'API sur une plateforme cloud gratuite (par exemple Render, Railway, Fly.io, ou une petite VM) à partir de l'image Docker Hub, et fournir l'URL publique.

5. Étapes suggérées (feuille de route)

Voici un déroulé recommandé ; vous pouvez l'adapter à votre méthode de travail.

#	Étape
1	Créer un dépôt GitHub vide pour le TP et y copier le projet fourni (system_metrics_agent).
2	Créer un compte Docker Hub et un token d'accès (Account Settings → Security → New Access Token).
3	Écrire et tester localement le Dockerfile.dev (build + run + vérifier le rechargement à chaud).
4	Écrire et tester localement le Dockerfile de production (build + run + vérifier le healthcheck).
5	Écrire docker-compose.yaml et vérifier que les services api et agent communiquent entre eux.
6	Ajouter les secrets DOCKERHUB_USERNAME et DOCKERHUB_TOKEN dans les paramètres du dépôt GitHub.
7	Écrire le workflow .github/workflows/ci-cd.yml (build → test → push conditionnel).
8	Pousser sur GitHub et vérifier l'exécution du pipeline dans l'onglet Actions.
9	Vérifier la présence de l'image sur Docker Hub, puis faire un déploiement de test via docker compose pull/up.
10	Rédiger le README.md et nettoyer le dépôt (suppression des fichiers inutiles, .gitignore correct).

6. Livrables à rendre

Vous devez remettre **exclusivement** les éléments suivants :

6.1 Un dépôt GitHub

Le lien vers un dépôt GitHub contenant :

- Le code source de l'application (non modifié dans sa logique métier).
- `Dockerfile.dev` et `Dockerfile` à la racine du projet.
- `docker-compose.yaml` (et éventuellement son override de développement).
- Le dossier `.github/workflows/` contenant votre pipeline CI/CD.
- Un fichier `.env.example` à jour (jamais de vrai `.env` commité).
- Un historique de commits **propre et incrémental** (pas un unique commit "projet final").

6.2 Un fichier README.md complet

Le README doit être rédigé avec soin et contenir au minimum les sections suivantes :

- **Présentation** du projet et de son architecture (bref rappel).
- **Prérequis** pour exécuter le projet (Docker, Docker Compose, comptes nécessaires).
- **Lancer en développement** : commandes exactes utilisant `Dockerfile.dev`.
- **Lancer en production** : commandes exactes utilisant `docker-compose` (build local puis, séparément, à partir des images Docker Hub).
- **Pipeline CI/CD** : explication du déclenchement, des étapes (build/test/push) et des secrets requis.
- **Images Docker Hub** : lien direct vers votre/vos dépôt(s) d'images publiées.
- **Choix techniques et difficultés rencontrées** : quelques lignes justifiant vos décisions (image de base choisie, gestion des deux services, etc.).
- **Captures d'écran ou logs** attestant du bon fonctionnement (pipeline vert, conteneurs actifs, appel API réussi).

6.3 Preuve de publication Docker Hub

Le lien public vers votre profil ou vos dépôts d'images Docker Hub doit être indiqué dans le README, et les images doivent être accessibles.

6.4 Modalités de soumission

Déposez le lien de votre dépôt GitHub sur la plateforme indiquée (ou envoyez-le par e-mail selon les consignes du cours), avant la date limite communiquée en séance. Aucun fichier compressé n'est attendu : seul le lien du dépôt (et éventuellement du profil Docker Hub) est requis.

7. Barème indicatif (/20)

Critère	Points
Dockerfile.dev fonctionnel (build + hot-reload)	3
Dockerfile de production (multi-stage, non-root, healthcheck, image optimisée)	4
docker-compose.yaml (services, réseau, dépendances, configuration)	3
Pipeline GitHub Actions (build, test, push conditionnel, secrets)	5
Publication Docker Hub + déploiement démontré	2
README.md complet et clair	2
Qualité générale (structure du dépôt, .gitignore, historique de commits)	1
Total	20

Toute image publiée involontairement avec des identifiants ou secrets en clair (dans le code, l'historique Git ou l'image Docker elle-même) entraînera une pénalité, quel que soit le reste du travail.

8. Ressources utiles

- Documentation Docker — <https://docs.docker.com>
- Documentation Docker Compose — <https://docs.docker.com/compose/>
- Documentation GitHub Actions — <https://docs.github.com/actions>
- Docker Hub — <https://hub.docker.com>
- Actions officielles Docker pour GitHub Actions — `docker/login-action`, `docker/build-push-action`
- Documentation FastAPI — <https://fastapi.tiangolo.com>